Year Up United
Data Analyst Training Academy: Introduction to Python

Week 5 Student Guide

This courseware is to be distributed to accompany instructor led training.

Copyright © 2026 Pluralsight & The Reliable Collaboration Company

Table of Contents

Week 5 Learning Objectives 4
Week 5 Learning Outcomes 5

MODULE 1 INTRODUCTION TO PYTHON 6

SECTION 1.A PYTHON BASICS 7
1.A.1 Intro to Python 8
1.A.2 Python code structure 9
1.A.3 Using `print()` to display a message 10
1.A.4 Using variables 11
1.A.5 `# Comments` in Python 12
1.A.6 Data types 13
1.A.7 Checking a value's data type 14
1.A.8 Reading error messages 15
1.A.9 Error types 16

SECTION 1.B CODING TOOLS FOR PYTHON 18
1.B.1 Where will we be working with Python? 19
1.B.2 Visual Studio Code 20
1.B.3 Installing extensions 21
1.B.4 Jupyter Notebook 22
1.B.5 Documentation and getting help 23

MODULE 2 BUILDING EXPRESSIONS 24

SECTION 2.A WRITING PYTHON EXPRESSIONS 25
2.A.1 Expressions 26
2.A.2 Arithmetic operators 27
2.A.3 Examples: Using arithmetic operators 28
2.A.4 Precedence 29
2.A.5 Constructing your code 31

SECTION 2.B USING MATH FUNCTIONS 32
2.B.1 Mathematical functions in Python 33
2.B.2 Using the `math` library 34
2.B.3 Many more modules! 35
2.B.4 Data types in mathematical expressions 36
2.B.5 Rounding numbers 37
2.B.6 Assignment operators 38
2.B.7 Precedence (again) 39

SECTION 2.C WORKING WITH STRINGS 40
2.C.1 Strings vs. numbers 41
2.C.2 Formatting strings with f-strings 42
2.C.3 More on using f-strings 43

MODULE 3 WORKING WITH DATA TYPES 44

SECTION 3.A CONVERTING DATA TYPES 45
3.A.1 Python is dynamically typed 46
3.A.2 Operations may affect data type 47
3.A.3 String manipulation: slicing 48
3.A.4 String manipulation: splitting 49
3.A.5 More string methods 50
3.A.6 Using the input() function 51
SECTION 3.B COLLECTION DATA TYPES 53
3.B.1 What are collections? 54
3.B.2 Lists 55
3.B.3 List functions 56
3.B.4 Tuples 57
3.B.5 Unpacking tuples and lists 58
3.B.6 More working with tuples 59
3.B.7 Sets 60
3.B.8 Set functions 61
3.B.9 Dictionaries 62
3.B.10 Dictionary functions 63
3.B.11 More dictionary functions 64
3.B.12 When are different collection types used? 65

MODULE 4 CONDITIONAL LOGIC & FLOW CONTROL 66

SECTION 4.A CONDITIONALS (`IF`, `ELIF`, `ELSE`) 67
4.A.1 The logic of `if, elif, else` 68
4.A.2 Comparison operators 69
4.A.3 Using Booleans 70
SECTION 4.B MORE CONDITIONAL LOGIC 71
4.B.1 Using `and` / `or` / `not` 72
4.B.2 Using `match case` statements 73
SECTION 4.C LOOPS 74
4.C.1 Using Loops 75
4.C.2 `for` loops 76
4.C.3 Using `enumerate()` 77
4.C.4 Loops and unpacking lists 78
4.C.5 `while` loops 79
4.C.6 Week 5 Wrap-Up 80

Week 5 Learning Objectives

- This week you will learn:
  - ◊ Setting up and using VS Code to write, run, and debug Python scripts
  - ◊ Python's core data types: int, float, str, bool, and None
  - ◊ Arithmetic expressions, operator precedence, and output formatting with f-strings
  - ◊ String manipulation: slicing, splitting, stripping, and common string methods
  - ◊ Type casting between data types using int(), float(), str(), and bool()
  - ◊ Accepting and validating user input with input()
  - ◊ Importing and using functions from the math module
  - ◊ The four Python collection types: list, tuple, set, and dictionary
  - ◊ Conditional logic with if/elif/else, Boolean operators, and match/case
  - ◊ Iteration with for loops (over lists, ranges, and with enumerate()) and while loops with break and continue
  - ◊ Reading and interpreting Python error messages and common error types
  - ◊ Version-controlling Python work to GitHub using VS Code's built-in source control

Week 5 Learning Outcomes

- By the end of the week, you will be able to:
  - ◊ Write a working multi-step Python script from scratch given a word problem (e.g., gross pay calculator, tile estimator)
  - ◊ Predict the output of a given code snippet involving variables, operators, and control flow
  - ◊ Identify and fix a NameError, TypeError, or ValueError from a traceback
  - ◊ Choose the appropriate collection type for a given scenario and operate on it correctly

Module 1

Introduction to Python

Section 1.A

Python Basics

1.A.1 Intro to Python

- High level programming language
  - ◊ Dynamic
  - ◊ Interpreted
- Easy, readable
- Born in the Netherlands, invented by Guido van Rossum
- Current version is Python 3
  - ◊ Some legacy programs may still use Python 2
- Used for
  - ◊ Application development
  - ◊ Data analysis / machine learning
  - ◊ (Automation) Scripting
  - ◊ Internet of things
  - ◊ Glue language

1.A.2 Python code structure

- Values (number, variable, list, dictionary, etc.)
- Operators (assignment, comparison, arithmetic, logical)
- Return and tab/indentation used to define lines and code blocks
  - ◊ Unlike in SQL, indentation in Python is not optional formatting; it is an important part of the language syntax
  - ◊ A misplaced indent changes what the code does, and a missing indent causes an error
- Use of capitalization
  - ◊ As a common convention, most code should be in lower case
  - ◊ Two exceptions:
    - * Upper case names written with PascalCase are used for class definitions (e.g. `FoodTruck`)
    - * ALL_CAPS_WITH_UNDERSCORES is used for constants (e.g. `MAX_SIZE`)

1.A.3 Using `print()` to display a message

- First program!

- Open the Command Prompt from your computer's Start menu (search `cmd` in the Start menu to quickly find this)

- Initiate Python by typing in the command `py` and hitting enter

  ◊ Python not found? Go to Start > Settings > Apps > Installed apps, then search for python

    * If Python 3 is installed, click the ellipses icon (...) then Modify, and check the selected options. Check off all "optional features" plus the top 3 "advanced options" (excluding "install for all users"), and click Install to modify your installation
    * If Python is not installed, install the latest version from python.org

- Type in the following line of code:

```
print('Hello world!')
```

- Hit enter to run the program

- Check the output – it should say `Hello world!`

  ◊ `print()` is a function defined in the standard Python library that is used to display a value or result

1.A.4 Using variables

- What is a variable?
  - ◊ A reusable element used to store a value, e.g. `x`, `color`, `acct_bal`
- Defining variables in Python
  - ◊ A variable name must start with a letter or the underscore character (cannot start with a number)
  - ◊ A variable name can only contain alpha-numeric characters and underscores (A-Z, a-z, 0-9, and _)
  - ◊ Variable names are case-sensitive (e.g. `age`, `Age` and `AGE` are three different variables)
    - * Common convention in Python is to write variable names in lowercase with words separated by underscores (“snake case”), e.g. `f_name`
  - ◊ Examples

```
greeting_1 = "Hi"
n = 8.5
lottery_pick = [1, 2, 8]
result = ''
```

- Reserved words
  - ◊ Some words cannot be used to name variables because they are reserved keywords – Python uses these words as part of its language, so it cannot understand them as variable names (e.g. and, or, if, for, while)
  - ◊ For detail see https://flexiple.com/python/python-reserved-words

1.A.5 # Comments in Python

- Comments are used to leave notes in code to you or other programmers about how the code works

- In Python, comments are prefixed with the # symbol

  ◊ You can start a line with # so that the full line is a comment

  ```
  # This is a comment
  ```

  ◊ You can also use # after a piece of code, and the interpreter will ignore the rest of the text on the current line as a comment

  ```
  print("Hello!")  # This is an inline comment
  ```

- You may also sometimes see multi-line comments appear as unassigned strings, starting and ending with '''

  ◊ This was an old trick that many Python coders have used, but it is not favored in the current Python style guide

- Comments are visible to someone viewing the source code

  ◊ So don't write something you don't want seen by other users!

1.A.6 Data types

- Some common data types used in Python:
  - ◊ String – any text/characters enclosed by `' '` or `" "` or `''' '''`
  - ◊ Integer – a whole number
  - ◊ Float – a decimal number
  - ◊ Boolean – a binary value of True or False
  - ◊ `None` – represents the absence of a value (like `null` in SQL)

- Examples

```
word = "banana"      # a string
num = 12             # an integer
not_num = '12'       # a string
```

- You can perform mathematical operations on a number
  - ◊ Example

```
num2 = num + num     # num2 is now 24
word2 = word + num   # error! can't "add" 12
                     # to banana
```

- You can **concatenate** strings using the + operator
  - ◊ Example

```
word2 = word + str(num)  # str() casts as string
                         # word2 result: banana12
```

1.A.7 Checking a value's data type

- Remember when you were creating tables and columns in SQL, and you had to assign each column a specific data type for values that it would store? Well, Python is a little different!

- Python uses **dynamic typing** which means that it checks the value at runtime and assigns the type automatically
  - ◊ This process is called **type inference** and sometimes is referred to as **duck typing** (i.e. if it walks like a duck and quacks like a duck, it must be a duck)
  - ◊ By contrast, SQL uses explicit *schema-level* typing
  - ◊ Other programming languages like Java or C++ are *statically typed*, where the programmer must *declare* the data type for variables

- The function `type()` is a useful tool for helping you understand what your code is doing by identifying the data type of a given value
  - ◊ `type()` lets you inspect the data type of any value
  - ◊ Examples:

```
type(12)                # <class 'int'>
type('12')              # <class 'str'>
type(12.0)              # <class 'float'>
type(True)              # <class 'bool'>

thing_1 = 'banana'
type(thing_1)           # <class 'str'>
thing_2 = 0
type(thing_2)           # <class 'int'>
```

1.A.8 Reading error messages

- Error messages are not “bad” – errors are *information*
  - ◊ In an error, Python is telling you exactly what it noticed and where
  - ◊ Learning from error messages is one of the most important programming skills you can build!

- When Python encounters a problem, it prints a **traceback**, a report describing what happened and where
  - ◊ Example:

```
Traceback (most recent call last):
    File "my_script.py", line 4, in <module>
        print('Total: ' + total)
TypeError: can only concatenate str (not "int") to str
```

  - ◊ Start reading at the bottom, as the last line is the actual error message that tells you what went wrong
  - ◊ The lines above it show where in the code the problem occurred (the file name and line number)

- In the example above:
  - ◊ `TypeError: can only concatenate str (not "int") to str` → Python is telling you it cannot combine a string and an integer using +
  - ◊ `line 4` → the problem is on line 4 of the file
  - ◊ `print('Total: ' + total)` → this is the specific line flagged

1.A.9 Error types

- The most common error types you will encounter:

Error	What it means	Common cause
`Syntax Error`	Python can't parse the code	Missing colon, unclosed bracket, typo in a keyword
`IndentationError`	Code block is misaligned	Extra or missing spaces at the start of the line
`NameError`	Python doesn't recognize a name used in the code	Using a variable before it's defined, or a typo in a variable name
`TypeError`	Wrong data type for an operation	Trying to add (or concatenate) a string and a number without casting
`ValueError`	Right type, but an invalid value	e.g. int('twelve') – text can't be converted to an integer

- Debugging habit to build from day one: when you get an error, read the message before anything else
  - ◊ It usually tells you exactly what went wrong and exactly where
  - ◊ The line number Python gives you is where to look first
- Don't be discouraged when you see error messages!
  - ◊ Every programmer, at every level, encounters them frequently
  - ◊ Remember: errors are information!
- We will talk about tools for constructing your code to **handle** errors in Week 6

Section 1.B

Coding Tools for Python

1.B.1 Where will we be working with Python?

- We started by running some simple instructions in the CMD terminal, but you'll need additional tools to create and save longer programs

- There are many possible options for coding with Python

- We will focus on two:
  - ◊ Visual Studio Code
  - ◊ Jupyter Notebook

1.B.2 Visual Studio Code

- Visual Studio Code (or VS Code) is a free application where you can write, run, and debug your code all in one place

- It's designed specifically to make code more readable and make the process of debugging more manageable

  ◊ It highlights your syntax in different colors to make it easier to read, flags potential errors as you type, and lets you run your work-in-progress programs without ever leaving the application

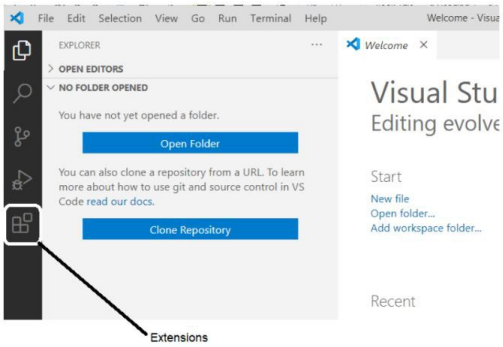
  ◊ VS Code and programs like it are often called IDEs (integrated development environments)

- VS Code is a highly flexible tool that supports coding in many different programming languages and is widely used by software developers

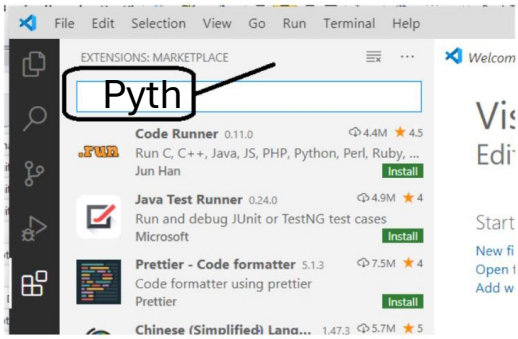
  ◊ We'll use VS Code this week to create and save Python files (.py)

1.B.3 Installing extensions

- To work with Python in VS Code, you'll need the Python extension
- To install extensions, click on the Extensions icon on the left bar



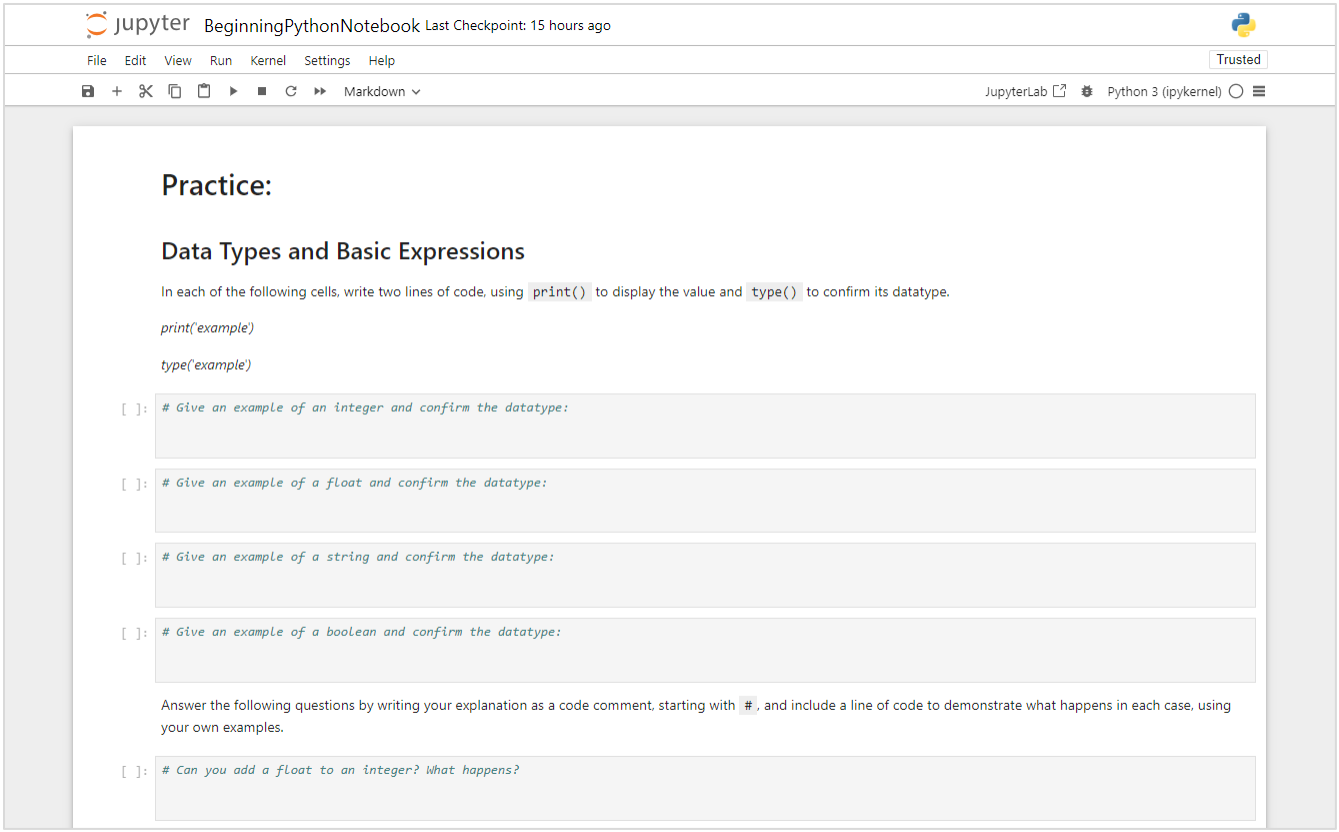
- In the search pane, type the name of the extension you want to install



- Then click the Install button
  - ◊ Note: the Python extension may already be installed on your machine
- You may have to close and then re-open Visual Studio Code to see your extensions

1.B.4 Jupyter Notebook

- Jupyter Notebook (and Jupyter Labs) can be used to create shareable and highly readable workbooks that combine code blocks and text



  - ◊ Jupyter Notebooks can also be read and edited in VS Code
  - ◊ Google also offers a version of notebooks via Google Colab

- This week we will focus on getting oriented with writing basic Python programs in VS Code, then next week we will dive into utilizing Jupyter Notebooks as we pivot toward data analysis

1.B.5 Documentation and getting help

- One of the most important meta-skills in programming (and in data analysis generally) is knowing how to look for and find information to help you with what you're trying to do
  - ◊ You are not expected to fully memorize every bit of the syntax that you learn in this course – even highly experienced programmers use reference materials to look things up!

- There are many resources for Python:
  - ◊ When using Python in the terminal, you can use its built-in `help()` function to look up information on using keywords, operators, modules, or other topics
  - ◊ The official Python documentation at docs.python.org is searchable and very comprehensive
  - ◊ w3schools, freeCodeCamp, and other learning resources provide well-organized, beginner-friendly references
  - ◊ When running a web search, a variation on "python how to [task]" returns useful results almost every time, although you may need to think carefully about how to apply those results to your specific problem
  - ◊ AI tools like ChatGPT, Claude, Gemini, etc. are generally very solid at providing code explainers and generating examples, but may sometimes be overly literal or may incorporate outdated concepts based on older training data

Module 2

Building Expressions

Section 2.A

Writing Python Expressions

2.A.1 Expressions

- An expression is any set of variables, literals, and operators that evaluate to a single value
  - ◊ Example
  - ◊ `# assuming num1 and num2 are the names of variables`
  - ◊
```
42
num1 + 1
(num1 + num2) / 2
```

- Expressions are often assigned to a variable
  - ◊ Example
  - ◊
```
num1 = 42
num2 = num1
nextValue = num1 + 1
average = (num1 + num2) / 2
```

- Over the next few pages, we will examine the arithmetic operators available to Python programmers

- We will examine other operators as well including comparison, logical, string, and more

2.A.2 Arithmetic operators

- The following operators are used to write mathematical expressions:

  `+` (addition)

  `-` (subtraction)

  `*` (multiplication)

  `/` (division, always returns a float)

  `//` (floor division, always rounds *down* to the nearest integer)

  `%` (modulo, finds the remainder)

  `**` (exponentiation, e.g. 2**3 returns 8)

2.A.3 Examples: Using arithmetic operators

- Example 1
  - ◊ `fuel_price = 120`
  - ◊ `sale_discount = .3`
  - ◊ `discount_price = (sale_discount) * fuel_price`
- Example 2
  - ◊ `total_minutes = 536`
  - ◊ `hours = total_minutes // 60`
  - ◊ `remaining_minutes = total_minutes % 60`
- Example 3 – String concatenation
  - ◊ `addr_num = '1233'`
  - ◊ `addr_st = 'Maple Ave'`
  - ◊ `addr_unit = ''`
  - ◊ `addr_full = addr_num + ' ' + addr_st + ' ' + addr_unit`
    - * What happens if instead we just use:

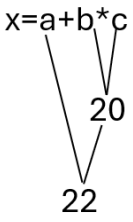
      ```
      addr_full = addr_num + addr_st +  addr_unit
      ```

2.A.4 Precedence

- Operator precedence describes the order in which operations are performed in an expression
  - ◊ Simplified version: parentheses → exponents → multiplication/division → addition/subtraction
    - * This is similar to the algebraic precedence that you might remember from math classes in school
    - * Complete order of operator precedence can be found at https://docs.python.org/3/reference/expressions.html#operator-precedence
  - ◊ Example

```
a = 2
b = 5
c = 4
x = a+b*c   # result is 22
```

- How did we come up with answer? Multiplication and division have higher precedence than addition and subtraction



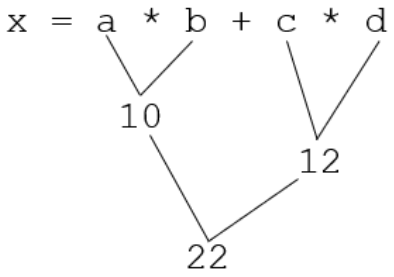
- Precedence can be changed by using parentheses
  - ◊ Example

```
x=(a+b)*c    # result is 28
```

- When operations have the same precedence, they are performed left to right
  - ◊ Example

```
a = 2
b = 5
c = 4
d = 3
x = a * b + c * d       # result is 22
```

- How did we calculate that result?



2.A.5 Constructing your code

- You may have noticed our examples and exercises so far:
  - ◊ Typically start by creating the desired variables
  - ◊ Then perform calculations using those variables in expressions
  - ◊ Lastly display the results
- This is a pattern that you should adopt as a habit when coding!
  1. Define known values
  2. Calculate the unknown
  3. Display the results

Section 2.B

Using Math Functions

2.B.1 Mathematical functions in Python

- You can do a lot of math using operators in Python, but there isn't an operator for everything you might want to do
  - ◊ No square root operator
  - ◊ No "rounding" operator

- Think about all those weird extra buttons on a scientific calculator



- Python has built-in **functions** that you can **call** in the expressions you write – sort of like those extra buttons on the calculator

- There are all sorts of functions, not just dealing with math, but math is where we're going to start

2.B.2 Using the `math` library

- Additional functions outside the standard Python library are included in modules which you can import and use
  - ◊ The keyword `import` is used to import a module
- The `math` module includes many useful mathematical functions
  - ◊ `.pi` produces the constant value pi
- Rounding functions included in `math`:
  - ◊ `.floor()` rounds a number value down to the integer
  - ◊ `.ceil()` rounds a number value up to the next integer
  - ◊ `.trunc()` rounds off the decimal part and leaves the integer, rounding toward zero (equivalent to floor() for positive numbers and ceil() for negative numbers)
- When calling a function from an imported module, the module name is included in front of the function
  - ◊ Example:

```
import math

seconds = 448
minutes = math.floor(seconds / 60)
```

2.B.3 Many more modules!

- While many mathematical functions are available in Python's core `math` module, for advanced calculations or enhanced processing power, you will often use a specialized library

- **Numpy** is a math library with expanded functionality for assigning number types, performing mathematical operations, and (of particular interest to data analysts!) creating and performing operations on arrays

- **Pandas** is used particularly for reading and analyzing data, and includes a very helpful construct known as a DataFrame

- **Matplotlib** is used to plot data graphically including bar charts, line charts, histograms, and many other visualizations

- **Scikit-learn** is a machine learning library that provides tools for building and evaluating predictive models, including regression, classification, and clustering, and is one of the most widely used libraries in data science workflows

- You may encounter additional libraries used for data science and computational applications down the road

2.B.4 Data types in mathematical expressions

- Primitives
  - ◊ Data held as a single element: an integer, float, string, Boolean, or None
- Compound data types
  - ◊ Collection data types that can hold multiple pieces of data, such as lists, tuples, sets, and dictionaries
    - * Some add-on Python libraries include their own collection types, such as NumPy arrays, or pandas Series and DataFrames

2.B.5 Rounding numbers

- The `round()` function
  - ◊ Value or variable to be rounded is specified first in parentheses
  - ◊ Optional second argument to specify number of places
  - ◊ Examples:

```
round(math.pi)     : 3
round(math.pi, 4)  : 3.1416
```

However:
`round(3.0, 4)` : `3.0` ← round() will not add trailing zeros

- Rounding to fixed decimal place using `format()`
  - ◊ Example:

```
format(math.pi, ".2f")   : '3.14'
format(3.0, ".2f")       : '3.00'
```

Note that this produces a string! Check by using the type() function

```
type(format(math.pi, ".2f")) : <class 'str'>
```

- Using floor division (`//`) or casting as integer, `int()`

- Using `math.floor()`, `math.ceil()`, or `math.trunc()`

2.B.6 Assignment operators

- These operators *assign* a value to a variable:

  `=` sets the variable to the given value

  `+=` adds the given value to variable's existing value

  `-=` subtracts the given value from the existing value

  `*=` multiplies the given value by the existing value

  `/=` divides the given value by the existing value

  `//=` divides the given value by the existing value (floor division)

  `%=` finds the remainder from dividing by the given value

2.B.7 Precedence (again)

- With what we know, our precedence chart has now expanded to include:

OPERATOR	ASSOCIATIVITY
( )	left-to-right
* / %	left-to-right
+ -	left-to-right
= += -= *= /= %=	right-to-left

Section 2.C

Working with Strings

2.C.1 Strings vs. numbers

- The string data type can include numbers, but you can't perform mathematical operations on a string.
  - ◊ 5 is a number
  - ◊ '5' is a string
  - ◊ What happens when you try print (5 * '5')
- You can easily cast a string into a number, or a number into a string, using constructor functions:
  - ◊ int()
  - ◊ float()
  - ◊ str()
  - ◊ bool()
- Note that when you cast a float as an integer, no rounding happens – the decimals just drop off

2.C.2 Formatting strings with f-strings

- There are several ways to build a string that combines text and variable values in Python
  - ◊ You have already seen two of them: using the + operator to concatenate strings, and using format() to control how a number is displayed
  - ◊ **f-strings** (short for **formatted string literals**) are the modern Python approach that brings these ideas together in a more elegant way

- How f-strings work: Prefix a string with the letter `f` and embed variable names and/or other expressions directly inside the string using curly braces `{}`
  - ◊ Syntax: `f'start text {variable_name} then more text'`

- Comparing the three approaches:
  - ◊ Starting data:
    ```
    name = 'Priya'
    score = 94.5
    ```
  - ◊ simple concatenation: requires casting non-strings
    ```
    print('Good work, ' + name + '! You scored ' + str(score))
    ```
  - ◊ using `.format()`: more flexible, but harder to read
    ```
    print('Good work, {}! You scored {}'.format(name, score))
    ```
  - ◊ f-string: clean and readable
    ```
    print(f'Good work, {name}! You scored {score}')
    ```

2.C.3 More on using f-strings

- Any Python expression can go inside the curly braces, not just variable names
  - ◊ Example:
    ```
    hours = 8
    rate = 18.50
    print(f'Total pay: {hours * rate}')
    ```
- f-strings also support the same format specifiers you saw in Section 2.B.5:
  - ◊ Example:
    ```
    import math

    print(f'Pi rounded to 2 places: {math.pi:.2f}')
    # output: 3.14

    print(f'Score: {score:.1f}%')
    # output: 94.5%
    ```
  - ◊ The `:.2f` syntax inside `{}` works exactly like it does in `format()` – it's the same formatting language, just written in one place
- f-strings were introduced in Python 3.6 and are now the standard approach
  - ◊ Going forward in this course, you will see f-strings used throughout, and you should default to this approach when you need to embed variables or non-string values in a string

Module 3

Working with Data Types

Section 3.A

Converting Data Types

3.A.1 Python is dynamically typed

- Python is a dynamically typed language
  - ◊ Some languages require types to be explicitly declared (strict type languages), but Python does not declare types as part of the variable
- Python detects the data type of values based on their format
  - ◊ `'12'` is a string (and so are `"12"` and `'''12'''`)
  - ◊ `12` is an integer
  - ◊ `12.0` is a float
  - ◊ `True` or `False` are Booleans
- Values may be cast as other data types using the appropriate function:
  - ◊ `int('12'): 12`
  - ◊ `str(12): ’12’`
  - ◊ `float(12): 12.0`
  - ◊ `int(12.0): 12`
  - ◊ `float('12'): 12.0`
  - ◊ `int('twelve'): ValueError: invalid literal for int()`

3.A.2 Operations may affect data type

- An expression that mixes integers and floats will result in a float
  - ◊ `12 * 3.0: 36.0`
- Division (/) always results in a float
  - ◊ `12 / 3: 4.0`
- But floor division (//) results in an integer
  - ◊ `12 // 3: 4`
- The % operator also results in an integer (where both operands are integers)
  - ◊ `15 % 4: 3`

    Consider: Why do // and % operations produce an integer?
- Results using the exponent operator (**) can be an integer or a float
  - ◊ `type(2**2): <class 'int'>`
  - ◊ `type(2.1**2): <class 'float'>`

3.A.3 String manipulation: slicing

- Strings can be **sliced** using the index of the string
  - ◊ Each character in a string is assigned an **index** number, starting with 0
    - * Example: The index of 'house' is:
      ```
      0 1 2 3 4
      h o u s e
      ```
  - ◊ The **negative index** counts backwards from the end, starting with -1
    - * Example: The negative index of 'house' is:
      ```
      -5 -4 -3 -2 -1
       h  o  u  s  e
      ```
  - ◊ A character of a string can be specified using its index (or reverse index) in square brackets
    - * Example: Where `var = 'house'`:
      ```
      var[1]: o
      var[0]: h
      var[-1]: e
      var[-2]: s
      ```
  - ◊ A slice of a string can be taken by specifying an index range, *up to but excluding* the second index number
    - * Example: Where `var = 'house'`:
      ```
      var[0:3]: hou
      var[:3]: hou
      var[2:4]: us
      var[2:-1]: us
      var[2:]: use
      ```
- To find the length of a string, use the `len()` function

3.A.4 String manipulation: splitting

- A string can be split into a list of strings based on a specified character using the `.split()` method
  - ◊ This is most commonly applied to strings that contain multiple words separated by spaces, or a text list of character-delimited items
  - ◊ If no delimiter is specified, the default is to split at each space
    - * Example:

```
string1 = “banana apple orange”
words1 = string1.split()
words1: ['banana', 'apple', 'orange']
```

  - ◊ A delimiter can be specified as an argument within the parentheses (must be formatted as a string)
    - * Examples:

```
string2 = "banana, apple, orange"
words2 = string2.split(',')
words2: ['banana', ' apple', ' orange']

string2 = "banana, apple, orange"
words3 = string2.split('a')
words3: ['b', 'n', 'n', ', ', 'pple, or', 'nge']
```

- We will talk more soon about **lists** and other collection types…

- Note: If you just want to remove white space from the beginning/end of a string, use the `.strip()` method
  - * `"  banana, apple, orange ".strip()`

3.A.5 More string methods

- A few other commonly used string methods worth knowing:
  - ◊ **`.upper()`** converts all characters to uppercase:

    `'hello'.upper()` → 'HELLO'
  - ◊ **`.lower()`** converts all characters to lowercase:

    `'Hello World'.lower()` → 'hello world'
  - ◊ **`.replace(old, new)`** swaps every occurrence of one substring for another:

    `'banana, apple, banana'.replace('banana', 'grape')` → 'grape, apple, grape'

    `'$1,250.00'.replace('$', '')` → '1,250.00'

- These are especially useful when cleaning data:
  - ◊ `.upper()` / `.lower()` can be used to standardize inconsistent capitalization (e.g., a user entering 'YES', 'Yes', or 'yes' can all be compared to 'yes' after calling `.lower()`)
  - ◊ `.replace()` removes or substitutes unwanted characters (e.g., stripping $ signs or commas from a numeric string before casting it as a float)

```
a = '€1.250,00'
a = a.replace('€', '')
a = a.replace('.', '')
a = a.replace(',', '.')
print(float(a))
```

3.A.6 Using the input() function

- The `input()` function asks the program user to input a value
  - ◊ This function accepts one optional argument which is the prompt displayed to the user; must be given as a string
  - ◊ When input() is assigned to a variable, the value that the user enters is then assigned to that variable
    - * Example:

```
x = input('Red or blue? ')
# if user enters 'red', now x = 'red'
```

- If you got to the bonus labs in Exercise 2.A, you may already have tried out using this useful and versatile function

- This is a powerful tool but has some pitfalls to watch for!
  - ◊ Because the value created by input() is always a string, if you want the user to input a number or Boolean, you must cast as the desired type
    - * Example:

```
y = int(input("How many cats do you have? "))
```

  - ◊ Case sensitivity can create issues (e.g. code expects 'Y'/'N', gets 'y')
    - * Control with `.upper()` / `.lower()` / `.title()` methods
  - ◊ Users can be unpredictable in their responses! Best practice is to test your code with multiple kinds of inputs

r/Jokes
u/Grievous_Nix • 2y

Join

A software tester walks into a bar.

Walks into a bar

Runs into a bar.

Crawls into a bar.

Dances into a bar.

Flies into a bar.

Jumps into a bar.

And orders:

a beer.

2 beers.

0 beers.

99999999 beers.

a lizard in a beer glass.

-1 beer.

"qwertyuiop" beers.

Testing complete.

A real customer walks into the bar and asks where the bathroom is.

Section 3.B

Collection Data Types

3.B.1 What are collections?

- So far we have been working with variables that store just one item at a time (a single string or number, aka *primitives*), but variables can also be used to store **collection** data types

- Python has four built-in data structures that can be used to hold a collection of objects:
  - ◊ List
  - ◊ Tuple
  - ◊ Set
  - ◊ Dictionary

- All of these collection types are **iterable** objects

- Each type of collection has different attributes and features which make them suitable for different purposes

- There are different built-in functions that work with each type – we will just look at a sampling

3.B.2 Lists

- Lists are used to store multiple items in a single variable, defined using square brackets, `[]`
  - ◊ Examples:

```
fruits = ['apple', 'peach', 'pear']
addresses = ['105 1st St', '8956 Rockford Blvd']
empty_list = []
```

- Attributes of a list:
  - ◊ Ordered
  - ◊ Changeable
  - ◊ Allows duplicate values
- List items are indexed
  - ◊ Just like with strings, indexing starts with 0 (and negative index with -1)

3.B.3 List functions

- Some useful built-in list functions and methods:
  - ◊ Find length of list:
    ```
    len(fruits)        # returns 3 for apple, peach, pear
    ```
  - ◊ Add to list:
    ```
    fruits.append('cherry')
    ```
  - ◊ Insert at an index:
    ```
    fruits.insert(1, 'pineapple')
    ```
  - ◊ Remove item at index:
    ```
    fruits.pop(1)
    ```
  - ◊ Remove item:
    ```
    fruits.remove('pear')
    ```
  - ◊ Sort the list:
    ```
    fruits.sort()
    ```
  - ◊ Reverse the items in a list:
    ```
    fruits.reverse()
    ```
  - ◊ Make a new list object of a list that's sorted:
    ```
    sorted(fruits)
    ```
  - ◊ Get the max value from the list:
    ```
    max(fruits)
    ```
  - ◊ Get the min value from the list:
    ```
    min(fruits)
    ```
  - ◊ Get the arithmetic sum of items in list:
    ```
    sum(fruits)         # TypeError with string values!
    ```
  - ◊ Keyword to test list membership:
    ```
    'cherry' in fruits # returns True
    ```

3.B.4 Tuples

- A tuple is a fixed collection of values defined within parentheses
  - ◊ Examples:

```
fruits = ('apple', 'peach', 'pear')
addresses = ('105 1st St', '8956 Rockford Blvd')
empty_tuple = ()
```

- Attributes of a tuple:
  - ◊ Ordered
  - ◊ Immutable (fixed, unchangeable)
  - ◊ Allows duplicate values
- Because it has a fixed value, a tuple is considered **immutable**
  - ◊ Overwriting entire tuple is possible, but it cannot be modified (e.g. new items appended or items removed)
  - ◊ Because they're immutable, tuples are a little bit faster in processing
  - ◊ Tuples can also be used to protect against undesired changes
- A tuple is indexed just like a list, starting with 0

3.B.5 Unpacking tuples and lists

- Tuples and lists (and other collection types) can be **unpacked**, that is, the values can be extracted out into separate variables

- Unpacking assigns each value to its own variable in a single step

  ◊ Example:

```
record = ('Laptop', 'East', 1200.00)
product, region, amount = record

print(product)   # Laptop
print(region)    # East
print(amount)    # 1200.0
```

- If you need some values but not others, use an underscore _ as a placeholder for any value you don't need to keep

  ◊ This is a Python convention that just means "I know there's a value here but I'm not going to use it"

```
_, region, amount = record   # product name not needed
print(region, amount)        # East  1200.0
```

- If the number of variables is less than the number of values in the collection, you can add an * to a variable name to assign the extra values to it as a list

  ◊ Example:

```
tuple_a = ("chocolate", "vanilla", "cherry", "mint")
flav1, flav2, *flavN = tuple_a
```

3.B.6 More working with tuples

- Once a tuple is created, you cannot change its values, as tuples are immutable (unchangeable) … but there are workarounds!
  - ◊ Convert the tuple into a list, change the list, then convert back to a tuple
    - * Example:

```
tuple_b = ("apple", "banana", "cherry")
tuple_b = list(tuple_b)
tuple_b.append("orange")
tuple_b = tuple(tuple_b)
```

  - ◊ You can also add/join tuples to tuples, so you can create a new tuple with additional values and add it to the original tuple
    - * Example:

```
tuple_c = ("lettuce", "carrot", "tomato")
y = ("cucumber",)   # That last comma makes this a tuple
tuple_c += y
```

  - ◊ You can also just assign a new tuple to the same variable name; this overwrites (rather than “changes”) the original tuple

3.B.7 Sets

- A set is an unordered collection of unique values defined within curly braces
  - ◊ Examples:

```
fruits = {'apple', 'peach', 'pear'}
addresses = {'105 1st St', '8956 Rockford Blvd'}
empty_set = set()
```

- Attributes of a set:
  - ◊ Unordered
  - ◊ Unchangeable, but items can be added or removed
  - ◊ Does not allow duplicates (values must be unique)
- If duplicated values are entered into a set, the value will only be stored once – de-duplication is automatic
- Because sets are unordered:
  - ◊ Sets are not indexed
  - ◊ Individual items cannot be modified (but can be added or removed)
  - ◊ Each time you print the set, the items may show up in a different order
  - ◊ Sets can be unpacked like lists and tuples, but the results are unpredictable as you can't know the order in which items will be assigned to given variables

3.B.8 Set functions

- You can also look up an item in a set by using the `in` keyword
  - ◊ Example:

```
ice_cr = {"chocolate", "vanilla", "cherry", "mint"}
print("mint" in ice_cr)
```

  - ◊ You can also combine the `not` operator with `in` to check if an item is *not* in the set

```
ice_cr = {"chocolate", "vanilla", "cherry", "mint"}
print("old car tires" not in ice_cr)
```

- You cannot change existing set items, but you can add new items using the `.add()` method
  - ◊ Example:

```
toppings = {"sprinkles", "caramel"}
toppings.add("hot fudge")
```

- To add items from another set (or other collection type!), use the `.update()` method
  - ◊ Example:

```
serv = {"cup", "cone", "waffle"}
size = ("single", "double")
serv.update(size)
```

3.B.9 Dictionaries

- A dictionary is a structured collection of key:value pairs and is defined by curly braces, {}
  - ◊ Key:value pairs are indicated with a colon and separated by comma
  - ◊ Often each key-value pair is written on its own line for readability, although this is not required
  - ◊ Example:

```
contact_bb = {
  "name": "Bilbo Baggins",
  "birthday": "1999-03-25",
  "email": "bilbo.baggins@email.mail"
  }

exam_1 = {"baggins":.87, "gamgee":.92, "took":.78}
```

  - ◊ An empty dictionary can be created using empty braces, `dict = {}`

- Attributes of a dictionary:
  - ◊ Ordered
  - ◊ Changeable
  - ◊ Does not allow duplicates (all keys must be unique)

- A dictionary item can be called / referred to using the key name
  - ◊ Example:

```
print(contact_bb["birthday"])
```

3.B.10 Dictionary functions

- Here is a sampling of built-in functions that can be used with dictionary objects:

- Retrieve key-value pairs, just keys, or just values:
  `dictionary_name.items()`
  `dictionary_name.keys()`
  `dictionary_name.values()`

- Retrieve a specific value by key name:
  `dictionary_name.get("name1")`

  ◊ If you use `.get()` to look up a key that does not exist in the dictionary, Python returns `None` rather than raising an error:

  ```
  menu = {'coffee': 3.50, 'tea': 2.75}
  print(menu.get('coffee'))    # 3.5 (value for key 'coffee')
  print(menu.get('juice'))     # None (key not found)
  ```

  With `.get()` you can also provide a default value as a second argument, which is returned instead of None when the key is missing:

  ```
  print(menu.get('juice', 'not found'))  # prints 'not found'
  ```

  ◊ In the scenario of trying to look up a missing key, the `.get()` method works a little better than simply pointing to the key using the syntax, e.g., `menu['juice']`, which would raise a KeyError

3.B.11 More dictionary functions

- Add a new dictionary item
  `menu['juice'] = 3.50`

- Update the dictionary with the items from the given argument:
  `menu.update({'juice':4.25})`

  ◊ The argument must be a key:value pair, or an iterable object with key:value pairs, or a dictionary

  ◊ Also can assign new value using `dictionary_name[key] =`

- Remove the last inserted item from the dictionary
  `menu.popitem()`

- Remove an item using a specified key name
  `menu.pop('juice')`

  ◊ Alternatively, delete using keyword **`del`** `dictionary_name[key]`

  ◊ Note that with no specified key, `del dictionary_name` will delete the dictionary completely!

- Check dictionary membership with `in`

```
'coffee' in menu     # True
'smoothie' in menu   # False
```

3.B.12 When are different collection types used?

- All of these collection types are worth knowing about, and you may encounter instances when each of these are helpful
  - ◊ Use a list when you need an ordered sequence of values you may want to change
  - ◊ Use a dictionary when you need to look up values by name, like a small table with a key column
  - ◊ Tuples are like lists, but fixed, so they are useful when performing operations where values should not change
  - ◊ Sets automatically remove duplicates, which is particularly useful for quickly finding unique values
- In general, the workhorse most-used collection types for data analysts are lists and dictionaries

Module 4

Conditional Logic & Flow Control

Section 4.A

Conditionals
(`if`, `elif`, `else`)

4.A.1 The logic of `if, elif, else`

- Up until now, all our logic has been sequential
  - `Step 1, step 2, step 3!`

- In Python, you can use an `if` statement to perform actions only when specified conditions are true

  ◊ Syntax

```
if condition:
    resulting expression
```

- To build the condition, you have to use **comparison operators** to compare how given values relate to each other

- If the condition is met, the indented block of code will be executed

- Code to run if the condition is *not* met can be specified with `else`

- Multiple alternative conditions can be provided using the `elif` block

  ◊ Example

```
if x < y:
   print("x is smaller")
elif x == y:
   print("x and y are the same")
else:
   print("x is bigger than y")
```

4.A.2 Comparison operators

- The more commonly used comparison operators are:
  == checks if the values are equal / the same
  != checks if the values are *not* equal / not the same
  > checks if the first value is greater than the second
  >= checks if the first value is greater than or equal to the second
  < checks if the first value is less than the second
  <= checks if the first value is less than or equal to the second

  ◊ Example: Ticket calculator

```
num_kids = int(input("How many kids to admit? "))
price = num_kids * 9.95          # price is $9.95/kid
if num_kids >= 6:
    price = price * .9           # a 10% discount
    print(price)
else:
    print(price)
```

  ◊ Example: Sales tax calculator

```
sale_amt = float(input("Sale amount: "))
tax = round(sale_amt * .08, 2)   # 8% sales tax
tax_exempt = input("Tax exempt? (Y/N) ")
if tax_exempt == 'Y':
     tax = 0
print(tax)
```

4.A.3 Using Booleans

- Boolean is a data type with only two possible values
  - ◊ `True` (equivalent to 1)
  - ◊ `False` (equivalent to 0)
- A Boolean is always returned when using a comparison operator
  - ◊ `4 < 10` returns `True`
  - ◊ `'banana' == 'grape'` returns `False`
- You can cast a value into a Boolean using the function `bool()`
  - ◊ You can use `bool()` on pretty much anything to reduce it to a value of True or False
    - * So, when do you think it will return `True`? And when `False`?
    - * `False` is returned for:
      `0`
      `0.0`
      `''` (empty string)
      `[]` (empty list)
      `{}` (empty dict)
      `()` (empty tuple)
      `None`
      `False` itself
    - * Everything else cast as a Boolean returns `True`

Section 4.B

More Conditional Logic

4.B.1 Using `and` / `or` / `not`

- The Boolean operators `and`, `or`, and `not` can be used to compare the relationship between multiple conditions in an `if` statement

  `and` (logical AND) – returns True if both conditions are met

  `or` (logical OR) – returns True if either condition is met

  `not` (negation) – returns True if the conditions do not match

- The precedence of Boolean operators is `not` > `and` > `or` (meaning `not` is the highest precedence)

  ◊ Example:
  ```
  print(True and False) : False
  print(True or True and False) : True
  ```

  ◊ Use parentheses to change evaluation order
  ```
  print((True or True) and False) : False
  ```

  ◊ A more realistic example, used in an `if` statement:

  ```
  strawb = input('Strawberry? Y/N: ')
  blueb = input('Blueberry? Y/N: ')
  fresh = input('Are the berries fresh? Y/N: ')
  if (strawb =='Y' or blueb =='Y') and fresh == 'Y':
      print('Buy it!')
  else:
      print("Don't buy it")
  ```

4.B.2 Using `match case` statements

- The `match case` statement is a powerful tool for pattern-matching
  - ◊ Example:

```
greeting = 'Hello, World!'
match greeting:
    case 'Hello, World!':
        print('Hello to you too!')
    case 'Goodbye, World!':
        print('See you later')
    case other:
        print('No match found')
```

- Basic `match case` statements look a lot like an `if` statement
  - ◊ For anyone with some background in other languages like Java or C, you may notice that `match case` looks a lot like a `switch` statement

- The concluding `case other` is equivalent to `else` in an `if-elif-else` statement
  - ◊ This can also be written as `case _`

Section 4.C

Loops

4.C.1 Using Loops

- Loops let you perform operations on a series of items

- There are two types of loops : **for** and **while**

  ◊ **`for`** loops run a fixed number of times based on an iterable, like a list or a range, so that the loop runs once *for* each item in the iterable

    * Syntax:

```
for i in iterable:
    operation to perform
    second operation to perform, etc.
```

  ◊ **`while`** loops continue looping *while* a condition is met; when the condition is no longer met, the loop exits

    * Syntax:

```
while condition:
    operation to perform
    second operation to perform, etc.
```

- Notice in the syntax for loops, the initial for/while statement *must* end with a colon (:) and the subsequent code block is indented

- Loops can also contain if/elif/else blocks

4.C.2 `for` loops

- `for` loops over elements of an iterable, such as a list, range, or string

- An **iterable** is any object capable of returning its members one at a time, permitting it to be *iterated over*

  ◊ Example using **list**:

```
x = ['a','b','c']  # x is a list, indicated by []
for y in x:  # y is used as an iterator variable
    print(y)
```

  Output:

```
a
b
c
```

  ◊ Example using **range**:

```
for i in range(1,4):  # using range() function
    print(i, 'banana')    # i is our iterator here
```

  Output:

```
1 banana
2 banana
3 banana
```

- The keyword `break` can be used to exit the loop when a condition is met

```
sales = [120.00, 340.00, 89.00, 675.00, 210.00, 890.00]

for amount in sales:
    if amount > 500:
        print(f'Large transaction found: ${amount}')
        break     # only the first +500 transaction prints
```

4.C.3 Using `enumerate()`

- The built-in **`enumerate()`** function is used when looping over a list, when you need both the item and its index position

- `enumerate()` wraps the iterable and returns each item paired with its index as a tuple, which you can unpack directly in the for statement

  ◊ Example:

```
fruits = ['apple', 'peach', 'pear']
for i, fruit in enumerate(fruits):
    print(i, fruit)
```

  Output:
  0 apple
  1 peach
  2 pear

- By default the index starts at 0, but you can start from any number using the optional `start=` parameter:

  ◊ Example:

```
for i, fruit in enumerate(fruits, start=1):
    print(f'{i}. {fruit}')
```

  Output:
  1. apple
  2. peach
  3. pear

- This is much cleaner than manually creating and incrementing a counter variable alongside your loop

4.C.4 Loops and unpacking lists

- When working with data, it's quite common that the iterable you're looping over is a collection of collections, such as a list of tuples where each tuple represents a single record
  - ◊ Example:

```
ice_cream_sales = [
    ('cone-single', 'chocolate', 4.00),
    ('cone-double',  'mint',  6.00),
    ('dish-single', 'cherry chip',  4.00),
]
```

- When looping over a list of tuples, you can unpack each tuple directly in the `for` statement
  - ◊ This keeps your loop easy to read and understand, as instead of writing index references (e.g. record[0], record[1], record[2]), you give each value a meaningful name right from the start
  - ◊ Example:

```
for item, flavor, price in ice_cream_sales:
    print(f'Sold a {flavor} {item} for ${price:,.2f}')
```

    Output:
    Sold a chocolate cone-single for $4.00
    Sold a mint cone-double for $6.00
    Sold a cherry chip dish-single for $4.00

- This is the same unpacking you saw in Section 3.B.5, occurring once per loop iteration rather than once for a single tuple

4.C.5 `while` loops

- `while` repeats a block of code as long as a certain condition is met

- The keyword `continue` can be used to skip to the next loop if a certain condition is met

  ◊ Example:

```
i = 0                  # set starting value for i
while i < 20:          # condition for the loop
    i += 1             # adds 1 to i with each loop
    if i == 13:        # sets condition for "continue"
        continue
    print(i)           # prints current value of i
```

- As with `for` loops, the keyword `break` can be used to break out of the `while` loop if a certain condition is met

  ◊ Example:

```
bank_bal = 1000           # set starting value

while bank_bal > 200:     # condition for the loop
    bank_bal -= 100
    if bank_bal == 500:   # sets condition for "break"
        break
    print(bank_bal)
```

4.C.6 Week 5 Wrap-Up

- This week you built the foundation of Python programming:
  - ◊ Module 1 introduced Python's structure, the tools you'll use to write code (VS Code and Jupyter Notebook), and the habit of reading error messages as information
  - ◊ Module 2 covered expressions, arithmetic, math functions, and f-strings, which form the building blocks for writing calculations and displaying results
  - ◊ Module 3 explored Python's data types and collection types -- lists, tuples, sets, and dictionaries -- and how to work with and manipulate them
  - ◊ Module 4 introduced the logic that makes programs dynamic: conditionals to make decisions, and loops to process collections of data one item at a time
- In Week 6 you will put these foundations to work:
  - ◊ Writing reusable functions to organize your code
  - ◊ Using classes to model real-world objects like sales records
  - ◊ Working with files so your programs can read and write data
  - ◊ Beginning to use Jupyter Notebook and getting started with NumPy as you pivot toward data analysis in Python

End of Week 5 Student Guide